Запасные и исследовательские идеи для алгоритмических задач

Рабочий резерв — финальная версия 5

Идеи, которые пока не входят в основной сборник, но заслуживают сохранения, дальнейшего исследования или могут послужить источником для новой формулировки.

Изменения относительно версии 4. Резерв синхронизирован с финальным основным сборником из шести задач. Добавлены четыре задачи, которые ранее входили в основной сборник, но были удалены после усиленного prior-art audit: Longest Cyclic Path in a Binary Tree, Perfect Square Product Pairs, Illuminated Terrain и Count Drawn Squares of All Sizes. Для них зафиксированы причины удаления и ближайшие структурные коллизии. Также обновлён статус Minimum Cost to Square Every Product после обнаружения близкого алгоритмического скелета в LeetCode 2862. Count Unique XOR Pairs остаётся сильной запасной задачей, выведенной из main collection по эстетическим, а не по correctness/originality причинам. Этот документ теперь служит финальным рабочим архивом идей на момент отправки портфолио.

1. Minimum Cost to Square Every Product

Статус: отклонена как main candidate / high originality risk

Дан массив целых чисел. Можно выбрать некоторые индексы и заменить выбранное значение на любое целое число; стоимость замены элемента равна абсолютной величине его исходного значения. Требуется добиться того, чтобы произведение любой пары элементов было полным квадратом, с минимальной суммарной стоимостью.

Ключ: для ненулевых чисел важен signed square-free kernel. Если оставить без изменения несколько ненулевых элементов, все они должны иметь один и тот же signed kernel. Для выбранного kernel K выгодно оставить все элементы этого класса, а остальные заменить; нули можно не менять.

Отсюда кандидат на решение: total_abs_sum - max_K(sum of |a_i| over elements with key K).

Почему не возвращать в основной сборник: Базовая signed square-free-kernel характеристика уже стандартна, а после prior-art audit обнаружилась более сильная структурная коллизия с LeetCode 2862 Maximum Element-Sum of a Complete Subset of Indices: compute square-free class -> sum weights by class -> keep the heaviest class. В этой постановке minimization лишь дополняет тот же максимум до total_abs_sum, а произвольная replacement-операция функционально эквивалентна weighted deletion, поскольку заменённое значение можно сделать совместимым с выбранным классом. Идея сохраняется как историческая ветка, но не как новая задача для LeetCode.

2. Unit Squares Covered by a Walk

Статус: историческая простая sibling-задача / не кандидат на публикацию

Дана строка Path из U, D, L, R, задающая путь по квадратной решётке. Нужно посчитать число единичных квадратов, все четыре стороны которых были когда-либо пройдены, не обязательно подряд.

Базовое решение: хранить пройденные рёбра; каждое новое ребро относится к двум соседним клеткам и обновляет их 4-битную маску. Клетка засчитывается, когда маска впервые становится 1111. Ожидаемое O(n) время и O(n) память.

Почему отложена: Как самостоятельная реализация задача корректна и минималистична, но сам объект «посчитать замкнутые квадраты по набору нарисованных рёбер» имеет старый prior art.

Кроме того, механизм существенно слабее версии про квадраты всех размеров. Сохраняется только как ранняя ступень развития геометрической ветки.

3. Unit Triangles Covered by a Walk

Статус: альтернативная версия предыдущей

Плоскость разбита на правильные единичные треугольники. Путь задаётся шестью направлениями движения по рёбрам решётки. Требуется посчитать полностью обведённые единичные треугольники.

Можно использовать целочисленную систему координат на треугольной/гексагональной решётке и 3-битную маску сторон каждого треугольника.

Почему отложена: Интересна геометрическая модель, но алгоритмический механизм почти тот же, что у Unit Squares Covered by a Walk.

4. Shadowed Area in Illuminated Terrain

Статус: derivative follow-up к удалённой Illuminated Terrain

Вместо длины освещённой верхней поверхности требуется найти суммарную площадь области над столбцами, находящейся в тени, не выходя за горизонтальные границы массива.

Используется та же граница тени при проходе справа налево. Для каждого столбца можно вычислить локальную площадь затенённой трапеции; дополнительный массив не требуется.

Почему отложена: Локальная геометрия площади тени остаётся интересной, но базовая Illuminated Terrain после affine-преобразования сводится к известной suffix-maximum задаче LeetCode 1762. Поэтому follow-up хранится только как исследовательская производная, а не как кандидат на новый основной problem statement.

5. Citizens and Friendship

Статус: research / требует изменения постановки

Дан массив ID. Два человека могут дружить, если сумма их ID делится хотя бы на одно из 3, 5 или 7. Требуется выбрать максимальное множество, в котором любые два человека могут дружить.

Первый инсайт: совместимость зависит только от остатка modulo 105. После сжатия получается maximum weighted clique на специальном графе из 105 типов. Инверсия условия лишь меняет clique на independent set и не устраняет трудную часть.

Почему отложена: Есть хороший первый шаг, но нет чистого второго. Нужна другая формулировка или дополнительная структура, после которой специальный граф действительно решается элегантно.

6. Three-Wick Burning

Статус: research

Даны положительные длины фитилей. За раунд выбираются ровно три положительных фитиля и одновременно поджигаются. Они горят с одинаковой скоростью до тех пор, пока самый короткий не сгорит полностью; после этого остальные два гаснут. Процесс повторяется, пока остаётся хотя бы три положительных фитиля. Цель — минимизировать суммарную длину остатков.

Для a <= b <= c операция равна (a,b,c) -> (0,b-a,c-a). Жадность «брать три самых длинных» неверна: пример [1,2,2,2,2] позволяет получить 0, хотя такая жадность оставляет положительный остаток.

Почему отложена: Для двух фитилей возникает близость к Last Stone Weight II. Для трёх пока не найден чистый инвариант или глобальное переописание; задача может оказаться многомерным knapsack без нужной эстетики.

7. Blocking Every Five-in-a-Row

Статус: reserve / proof structure пока не устраивает

На m x n доске нужно заранее поставить минимальное число X так, чтобы каждый горизонтальный, вертикальный или диагональный отрезок из пяти последовательных клеток содержал хотя бы один X. Эквивалентно: остальные клетки можно считать O и максимизировать число O при запрете любой пятёрки O подряд.

Есть естественные периодические конструкции через раскраску modulo 5 и сильное ощущение, что оптимум должен иметь короткую формулу.

Почему отложена: Проверка показала, что строгий proof требует неприятного специального разбора, в частности вокруг размера 7 x 8. Такая «техническая хвостовая часть» противоречит эстетике основного сборника. Сохраняем идею, но не текущую доказательную форму.

8. Maximum Two-Leg Geodesic on a Cube Surface

Статус: research

Дано множество точек на поверхности куба. Требуется выбрать три точки A, B, C так, чтобы максимизировать d_surface(A,C) + d_surface(C,B), где d_surface — кратчайшее расстояние по поверхности куба.

Расстояние между фиксированной парой точек можно вычислять через конечное число развёрток граней. В версии, где точки разрешены только в восьми вершинах куба, задача схлопывается почти в brute force и слишком проста; интерес представляет именно версия с произвольными точками на поверхности.

Почему отложена: Локальная геометрия понятна, но не найден алгоритмически сильный способ избежать тяжёлого перебора по тройкам/парам для большого n.

9. Hexagonal Plane Section of a Cube

Статус: research

Даны n точек на поверхности куба. Требуется определить, существуют ли три неколлинеарные точки, плоскость через которые пересекает куб невырожденным шестиугольником.

Для куба [-50,50]^3 и плоскости с нормалью (a,b,c), d = n·A, найден простой критерий. Если S = |a|+|b| +|c| и M = max(|a|,|b|,|c|), то невырожденное шестиугольное сечение получается при |d| + 100M < 50S.

Почему отложена: Критерий для одной тройки красивый, но вопрос существования среди n точек пока даёт очевидный O(n^3) перебор. Не найден второй инсайт, который превратит его в полноценную алгоритмическую задачу.

10. Minimum Variance Reconstruction with Window 3

Статус: естественный harder follow-up к Minimum Variance Reconstruction

Дан массив M длины n-2, где M_i = A_i + A_{i+1} + A_{i+2} для неизвестного массива A из положительных целых чисел. Нужно вернуть любой A минимальной дисперсии.

Разность любых двух решений d удовлетворяет d_i + d_{i+1} + d_{i+2} = 0, откуда d_{i+3} = d_i. Поэтому все решения отличаются периодической поправкой (x, y, -x-y, x, y, -x-y, ...).

Положительность даёт целочисленную треугольную область для (x,y), а дисперсия становится выпуклой квадратичной функцией двух переменных.

Почему отложена: Очень естественное развитие принятой задачи с окном 2, но как отдельная задача сложнее и потенциально требует более тяжёлой двумерной целочисленной оптимизации. Сохранить как Part B / исследовательское направление, не усложняя основную Medium-задачу.

11. XOR Drafting with a Chosen Initial Value

Статус: research / новизна внешнего слоя требует проверки

Дан массив натуральных чисел. До начала драфта первый игрок выбирает натуральное число x, отсутствующее в массиве, и назначает его второму игроку как начальный XOR-score. Затем игроки по очереди забирают элементы массива, начиная с первого; каждый XOR-ит выбранное число со своим текущим score. Нужно найти минимальное x, при котором первый игрок может гарантировать победу, или -1.

Если T — XOR всего массива, а P — XOR элементов первого игрока, финальные значения сравниваются как P и P XOR (x XOR T). Для фиксированного x результат сравнения определяется старшим установленным битом x XOR T; далее игра схлопывается в бинарную parity-drafting игру.

Почему отложена: Версия без предварительного выбора x практически совпадает с известной Codeforces 1384D GameGame. Поэтому новый вклад должен находиться именно во внешней задаче «выбрать минимальное отсутствующее x»; пока не ясно, достаточно ли этот слой нов и содержателен.

12. Longest Cycle in an XOR-Interval Graph

Статус: очень перспективный research / bitwise + graph

Даны целые 0 <= a < b. Вершины графа — все k из [a,b]. Для разных k,l проводится ребро, если popcount(k XOR l) равно фиксированному числу r. Требуется длина самого длинного простого цикла.

Основной вариант: r = 2. Тогда parity popcount сохраняется, граф распадается на две части. На полном d-битном кубе это соответствует двум halved-cube компонентам. На малых интервалах экспериментально наблюдалось, что длина longest cycle совпадает с размером крупнейшего нетривиального biconnected block; возможная гамильтоновость таких блоков пока не доказана.

Альтернативный вариант: r = 3. Тогда parity popcount меняется на каждом ребре, поэтому граф двудольный, а любой цикл имеет чётную длину и не может быть длиннее 2*min(|E|,|O|). Эта версия имеет принципиально другую структуру и может оказаться как проще, так и сложнее r = 2.

Почему отложена: Statement почти идеален по минимализму и закрывает сразу два пробела портфолио — bitwise и graph. Но пока нет структурной теоремы или алгоритма для интервала [a,b] без явного построения огромного графа. Исследовать варианты r=2 и r=3 независимо.

13. XOR Square Membership Queries

Статус: запасная / bitwise + preprocessing; исторический предшественник Count Unique XOR Pairs

Даны два массива неотрицательных целых чисел M и N. Нужно посчитать индексы i массива N, для которых существует индекс j такой, что M[j] XOR N[i] является полным квадратом.

Пусть f[x] — число вхождений x в M, а g[s] = 1, если s является полным квадратом в выбранном битовом universe, и 0 иначе. Тогда h[x] = sum_y f[y] * g[x XOR y]. Условие h[x] > 0 эквивалентно существованию элемента M[j], для которого M[j] XOR x — полный квадрат. Массив h является XOR-convolution f и g и может быть вычислен сразу для всех x через FWHT; после preprocessing каждый N[i] проверяется за O(1).

Историческая ветка идеи: исходно предполагалось условие «XOR является степенью двойки». Оно оказалось слишком простым, потому что означает отличие ровно в одном бите и сводится примерно к 30 hash-lookups на запрос. Случайная замена степеней двойки на полные квадраты обнаружила более глубокий механизм XOR-convolution, который затем привёл к более естественной постановке Count Unique XOR Pairs.

Почему отложена: Сама задача чистая и содержательная, но свойство «полный квадрат» почти не взаимодействует с XOR: для алгоритма это лишь заранее известное множество допустимых XOR-масок. Поэтому условие немного выглядит подобранным ради convolution. Count Unique XOR Pairs устраняет эту конкретную искусственность, потому что multiplicity XOR-значений возникает непосредственно из вопроса, но сама FWHT-ветка теперь также хранится в резерве из-за более общего эстетического замечания о специализированном transform как отдельном шаге решения.

14. Count Unique XOR Pairs

Статус: выведена из основного сборника / сильная запасная bitwise + preprocessing

Даны два массива неотрицательных целых чисел M и N. Для каждой пары индексов (i,j) определяется значение X_{i,j} = M_i XOR N_j. Пара называется уникальной, если никакая другая пара индексов не даёт того же XOR-значения. Требуется посчитать число уникальных пар. Пары различаются по индексам, поэтому повторяющиеся значения в массивах по-прежнему создают разные пары.

Ключевой переход: пусть f[x] и g[x] — частоты значений в M и N. Для каждого XOR-значения z число пар, которые его порождают, равно c[z] = sum_x f[x] * g[x XOR z]. Значит, искомый ответ — количество z, для которых c[z] = 1. Массив c является XOR-convolution частот f и g.

Оптимальное решение: при 0 <= M_i, N_j <= 10^6 использовать universe U = 2^20. Выполнить FWHT для f и g, перемножить преобразованные массивы покомпонентно и применить обратный FWHT с делением на U. Все вычисления можно делать точно в signed 64-bit integer. Сложность: O(|M| + |N| + U log U) времени и O(U) памяти.

Почему выведена из основного сборника: Первый инсайт задачи остаётся сильным и естественным — вместо перебора до 10^12 пар нужно посчитать multiplicity каждого XOR-значения. Но после появления формулы XOR-convolution возникает отдельный специализированный шаг: нужно знать или самостоятельно вывести Fast Walsh-Hadamard Transform. Даже при полном понимании и доказательстве FWHT этот переход ощущается как самостоятельная техника, «довешенная» к задаче, а не как неизбежное продолжение правильного взгляда на исходный объект. Для основного сборника, где особенно ценится непрерывная цепочка «переописание -> резкое упрощение», это делает задачу менее органичной. Она остаётся хорошей задачей на XOR-convolution и может быть уместна в более transform-oriented или contest-oriented наборе.

История: задача некоторое время входила в основной 10-задачный сборник и прошла полноценное оформление с examples, editorial, correctness summary, implementation details и проверкой 64-bit bounds. Перенос обратно в резерв связан не с ошибкой в алгоритме или недостатком оригинальности, а с пересмотром эстетического соответствия общей коллекции.

15. Longest Cyclic Path in a Binary Tree

Статус: выведена из основного сборника / high originality risk для LeetCode

Дано бинарное дерево со значениями 0, 1, 2. Требуется максимальная длина простого пути, значения которого глобально идут либо по циклу 0 -> 1 -> 2 -> 0 -> ..., либо по обратному циклу 0 -> 2 -> 1 -> 0 -> ... .

intended solution: для каждой вершины считать две downward-величины forward(v) и backward(v), соответствующие шагам +1 и -1 modulo 3. Лучший путь с верхней вершиной v склеивает две противоположные руки и имеет длину forward(v) + backward(v) - 1. Итеративный postorder даёт O(n).

Почему удалена из main collection: LeetCode 549 Binary Tree Longest Consecutive Sequence II использует практически тот же алгоритмический скелет: возрастающая и убывающая downward-ветви, два DP-состояния и объединение через текущую вершину формулой inc + dec - 1. Замена обычного +/-1 на +/-1 mod 3 меняет relation, но не центральный insight и recurrence. Как independently rediscovered variation задача приятна, но именно LeetCode её предлагать неудобно.

16. Perfect Square Product Pairs

Статус: выведена из основного сборника / direct prior-art collision

Дан массив целых чисел. Требуется посчитать пары индексов i < j, для которых M_i * M_j является полным квадратом; ноль считается квадратом. Для ненулевых чисел совместимость определяется равенством signed square-free kernels; нули считаются отдельно.

Оптимальное решение: sieve/SPF для square-free kernels, frequency map по signed kernel и отдельная формула для пар с нулями. Сложность O(V log log V + n log V) или лучше при предвычислении kernels до V.

Почему удалена из main collection: UVa 11191 Perfect Square уже содержит последовательность целых чисел со знаками и нулём и просит, в частности, посчитать пары с квадратным произведением. Наша задача фактически совпадает с pair-part старой UVa-задачи. Это прямой prior art, поэтому использовать её как оригинальную submission нельзя.

17. Illuminated Terrain

Статус: выведена из основного сборника / high originality risk

Дан массив положительных высот столбцов. Свет идёт сверху справа под углом 45 градусов. Нужно посчитать столбцы, у которых положительная часть верхней горизонтальной грани освещена; касание в одной точке не считается.

Геометрически столбец i освещён тогда и только тогда, когда H_i > max_{j>i}(H_j - (j-i)). Это можно поддерживать одним right-to-left shadow envelope за O(n) времени и O(1) памяти.

Почему удалена из main collection: после замены A_i = H_i - i условие становится A_i > max_{j>i} A_j — ровно strict suffix maximum / Ocean View condition. Это напрямую сводится к LeetCode 1762 Buildings With an Ocean View (разница — affine transform и count вместо списка индексов). Геометрическая оболочка естественна, но центральный алгоритмический скелет уже есть на LeetCode.

18. Count Drawn Squares of All Sizes

Статус: выведена из основного сборника / сильная вариация известного объекта

Дана строка walk по квадратной решётке. Каждый шаг рисует unit edge; повторные проходы не добавляют новое ребро. Нужно посчитать все axis-aligned квадраты положительного целого размера, у которых весь периметр нарисован.

Сильная часть решения: после дедупликации рёбер построить maximal horizontal/vertical runs и значения right[p], up[p]. Для каждого нижнего левого угла перебираются только s <= min(right[p], up[p]), а верхняя и правая стороны проверяются hash lookup. Для числа кандидатов T используется bound A_s <= min(E, E^2/(4s^2)); разбиение по s ~ sqrt(E) даёт T = O(E^{3/2}) = O(N^{3/2}), и fully drawn k x k grid показывает tightness для этой enumeration strategy.

Почему удалена из main collection: UVa 201 Squares уже задаёт набор горизонтальных и вертикальных unit edges и просит считать полностью построенные квадраты всех размеров. Наш walk после линейного preprocessing превращается именно в такой геометрический объект. Sparse coordinates и O(N^{3/2}) analysis являются содержательным улучшением алгоритма, но при строгом стандарте originality это scaled-up version давно известной задачи, а не новый problem statement. Сохраняется как одна из технически самых интересных rejected ideas.

Не включено в резерв

Minimum Knight Moves. Отклонена после обнаружения практически прямого аналога на LeetCode; дальнейшая доработка не планируется.

Примечание

Этот документ — финальный рабочий инвентарь проекта на момент отправки основного портфолио. Он намеренно смешивает сильные reserve-идеи, research-направления, superseded siblings и задачи, отклонённые из-за prior art. Наличие идеи здесь не означает, что её следует публиковать. Перед возвращением любой задачи в активную разработку нужно заново проверить формулировку, примеры, ограничения, оптимальное решение, краевые случаи и актуальный prior art.